Diffusion-based UAV obstacle avoidance: verification, borrowing, and v8.1 upgrade

Of 26 claimed diffusion works from two Chinese survey documents, 25 are verified real and 1 is fabricated (DiTP). Three of the real papers — DiffusionDrive, DPPO, and a point-cloud-guided energy-based diffusion paper (2507.09383) — offer directly transplantable components that address the iDP3 system's two core failure modes: obstacle ignorance in unseen worlds and multi-modal mode switching near goals. The single most impactful change for v8.1 is replacing pure-noise initialization with **anchor-based truncated diffusion** (from DiffusionDrive) combined with **classifier-free guidance for goal conditioning** and **CoDiG-style barrier gradients** for obstacle avoidance — a combination that simultaneously fixes the 2.24× goal-proximity variance, eliminates extreme yaw/altitude excursions, and enables 2–5 step denoising for real-time control.

The fabrication rate is surprisingly low (1/26), but several papers are misattributed to UAV contexts when they actually target image generation (D-Fusion, DDPO) or pedestrian prediction (Leapfrog). The surveys' real danger is not hallucination but **misleading relevance claims** that waste engineering effort.

---

1. Executive summary of verification and borrowing

The verification exercise reveals a landscape where most papers exist but few directly apply to 3D point-cloud UAV navigation in PX4/Gazebo. Only **6 of 25 verified works** have been tested on UAVs (SwarmDiffusion, ImpedanceDiffusion, DroneDiffusion, FlightDiffusion, SketchPlan, and the Nano-Drone paper), and only 3 of those use diffusion for trajectory planning rather than dynamics modeling or video generation. The gap between "diffusion for robotics" and "diffusion for UAV obstacle avoidance with point clouds" remains substantial.

For borrowing, the top-5 sources of transplantable components are: **(1) DiffusionDrive** (anchor-based truncated diffusion — Phase 1a backbone), **(2) CoDiG** (barrier-function guidance injected at denoising time — Phase 2 safety), **(3) Paper #20** (point-cloud-conditioned energy-based diffusion with APF — closest architectural match), **(4) DPPO** (RL fine-tuning of diffusion policies — Phase 2 fallback), and **(5) NavDP** (critic-based trajectory selection — addresses mode switching). The classifier-free guidance technique from BESO (not on the original list but well-established) is the single simplest fix for goal-proximity instability, requiring ~50 lines of code change and zero architectural modifications.

---

2. Full verification table

#	Name	Real?	Correct arXiv	Venue	Code
1	SwarmDiffusion	✅ Real	2512.02851	Preprint 2025	❌
2	ImpedanceDiffusion	✅ Real	2603.09031	Preprint Mar 2026	❌
3	DroneDiffusion	✅ Real	2409.11292	ICRA 2025	✅ github.com/avirupdas55/DroneDiff
4	FlightDiffusion	✅ Real	2509.14082	Preprint 2025	❌
5	NavDP	✅ Real	2505.08712	Preprint 2025	✅ github.com/wzcai99/NavDP
6	DiTP	❌ **Not found**	N/A	N/A	N/A
7	DiPPeR	✅ Real	2310.07842	ICRA 2024	✅ Project page
8	DiffusionDrive	✅ Real	2411.15139	**CVPR 2025** (Highlight)	✅ github.com/hustvl/DiffusionDriv
9	SwarmDiff	✅ Real	2505.15679	CVPR 2025 Workshop	❌
10	DiffuserLite	✅ Real	2401.15443	**NeurIPS 2024**	✅ CleanDiffuser library
11	SafeDiffuser	✅ Real	2306.00148	**ICLR 2025**	✅ github.com/Weixy21/SafeDiffus
12	MPD	✅ Real	2308.01557	**IROS 2023**	✅ github.com/joaoamcarvalho/mp public
13	DPCC	✅ Real	2412.09342	**L4DC 2025**	✅ github.com/ralfroemer99/dpcc
14	CoDiG	✅ Real	2505.13131	**CoRL 2025**	❌

#	Name	Real?	Correct arXiv	Venue	Code
15	MBD	✅ Real	2407.01573	Preprint 2024	✅ github.com/LeCAR-Lab/model-based-diffusion
16	DPPO	✅ Real	2409.00588	**ICLR 2025**	✅ github.com/irom-princeton/dppo
17	DDPO	✅ Real	2305.13301	Preprint 2023	✅ github.com/kvablack/ddpo-pyto
18	DEAL	✅ Real	OpenReview only	**NeurIPS 2025**	❌
19	D-Fusion	✅ Real	2505.22002	**ICML 2025**	❌
20	PC-Guided Energy Diffusion	✅ Real	2507.09383	**IEEE RA-L 2025**	❌
21	Aligning Diffusion + Constraints	✅ Real	2504.00342	Preprint/Springer 2025	❌
22	SketchPlan	✅ Real	2510.03545	IEEE RA-L 2025	✅ github.com/sixnor/SketchPlan
23	Nano-Drone OA	✅ Real	N/A (MDPI)	MDPI Drones 2024	✅ github.com/noahzn/DDND
24	Leapfrog Diffusion	✅ Real	2303.10895	**CVPR 2023**	✅ github.com/MediaBrain-SJTU/L
25	Hybrid AI Drone Nav	✅ Real	2501.04472	MDPI AI 2024	❌
26	Constrained Bridged Diffusion	✅ Real	2502.20371	**AAAI 2025**	❌

3. Hallucination analysis and why it matters

The one fabricated paper — DiTP ("Diffusion Transformer Planner") — does not exist under that name or acronym in any indexed database. The concept it describes (a DiT-backbone

planner) does exist in other forms: DiffuserLite uses a DiT backbone, and Zheng et al.'s "Diffusion Planner" (ICLR 2025) applies DiT to autonomous driving. The survey likely conflated these into a phantom paper with a catchy acronym. This matters because a team might waste days searching for "DiTP code" that doesn't exist. NeurIPS + 2

More insidious than outright fabrication are the **misleading relevance attributions**. Two verified papers — DDPO and D-Fusion — are **pure image generation works** that have nothing to do with robotics trajectory planning. DDPO (Black et al., Berkeley) fine-tunes Stable Diffusion for aesthetic quality using RL. D-Fusion aligns text-to-image diffusion with visual consistency preferences. If the surveys implied these are robotics trajectory methods, that is a conceptual hallucination even though the papers are real. Similarly, Leapfrog Diffusion Model (CVPR 2023) predicts pedestrian trajectories from overhead video — it does not generate robot actions and cannot be "borrowed" for UAV control. OpenReview

The **ImpedanceDiffusion arxiv ID "2603.09031"** was flagged as suspicious because "2603" looks like a future year, but this is legitimate — arxiv IDs encode submission month (March 2026), not the year in a 4-digit block. This is a Skoltech paper submitted March 10, 2026, just five weeks before the current date. The survey was correct. arXiv

Bottom line: 1 fabricated paper, 3 papers with misleading robotics relevance claims, 0 incorrect arxiv IDs among the papers that exist. The surveys are more trustworthy than expected for AI-generated content, but the relevance framing requires expert filtering.

---

4. Borrowing matrix

The table below maps each verified, relevant work against the user's v8 phases. Papers irrelevant to UAV trajectory planning (DDPO, D-Fusion, Leapfrog, Hybrid AI, Nano-Drone CNN) are excluded.

Work	Phase 0 (Data/Env)	Phase 1a (Truncated Diff)	Phase 1b (Gym wrapper)	Phase 2 (DPPO or CBF)	Effort	Expected benefit
DiffusionDrive	—	★★★ Core backbone	—	Informs anchor design	Medium	Very high - fixes mode switching, enables 2-step denoising
CoDiG	—	★★ Barrier guidance	—	★★★ Safety filter	Medium	High — obstacle avoidance without retraining
Paper #20 (PC-Energy)	★ PC augmentation ideas	★★★ Architecture template	—	★★ APF integration	High	Very high - closest architectu match
DPPO	—	—	★★★ RL infrastructure	★★★ Core fine-tuning	Medium	High — structured on-manifo RL
NavDP	★★ Data pipeline ideas	★★ Critic for selection	★ Env design	★ Critic training	High	Medium-high — generate-then-selec paradigm
SafeDiffuser	—	—	—	★★★ CBF framework	Medium	High — formal safe guarantees
DroneDiffusion	—	—	—	★ Dynamics model	Low	Low — different problem (dynamics not planning)

Work	Phase 0 (Data/Env)	Phase 1a (Truncated Diff)	Phase 1b (Gym wrapper)	Phase 2 (DPPO or CBF)	Effort	Expected benefit
SwarmDiffusion	★ Multi-embodiment data	★ FiLM conditioning	—	—	High	Low — RG based, not point cloud
ImpedanceDiffusion	—	★ Dual-planner idea	—	★ APF local control	Medium	Medium — Crazyflie-specific
SketchPlan	★★ 3DGS data pipeline	—	—	—	Medium	Medium — sim-to-real evidence
DPCC	—	—	—	★★ Constraint framework	Medium	Medium — differential constraints
MPD	—	★ Cost guidance	—	★ Constraint guidance	Low	Low — 2D/arm, no navigation
MBD	—	—	—	★ Dynamics-aware diff	Medium	Low — MuJoCo-oriented
DiffuserLite	—	★ Hierarchical planning	—	—	Medium	Low — D4 benchmark
Constrained Bridged Diff	—	★ Bridge construction	—	★ Constraint modeling	High	Low — theoretical AV-focused
Aligning Diff + Constraints	—	—	—	★ Constraint alignment	Low	Low — tabletop or

---

5. The five most impactful borrowing opportunities

5a. DiffusionDrive's anchor-based truncated diffusion eliminates mode switching

DiffusionDrive's core innovation is replacing pure-noise initialization with **anchored Gaussian**

distributions centered on K prototype trajectories extracted via K-means clustering. Instead of 20 DDIM denoising steps from random noise, the model performs **2 refinement steps** from the nearest anchor. This directly solves two of the user's problems simultaneously. (arXiv + 2)

The **mode-switching problem** (2.24× variance at goal proximity) arises because standard diffusion samples from a single isotropic Gaussian that covers all trajectory modes. Near the goal, the model oscillates between "approach from left" and "approach from right," producing averaged, physically invalid actions. Anchors partition the output space into distinct modes — each anchor captures one behavioral pattern. At inference, the scene encoder selects the most appropriate anchor, and the diffusion model only refines within that mode's neighborhood. The extreme yaw rotation (336°) becomes impossible because no anchor trajectory contains such behavior. (arXiv) (l-newcar)

Implementation for UAV: Cluster the 200 EGO planner episodes into **5–10 trajectory prototypes** in (x, y, z, yaw) space. Candidates: straight-to-goal, avoid-left, avoid-right, climb-over, decelerate-to-hover. Replace iDP3's pure-noise sampling with `x_init = anchor_k + σ·ε` where σ is small (equivalent to truncating the noise schedule to the last ~5% of timesteps). The cascade decoder's cross-attention with point cloud features replaces DiffusionDrive's BEV attention. Engineering effort is **medium** — the denoising loop changes are straightforward, but the anchor selection head (which scores anchors based on scene context) requires design. The **DiffusionDriveV2** extension (arXiv 2512.07745) adds RL-based negative-mode suppression that would further prevent collision trajectories. (arXiv) (arXiv)

5b. CoDiG's barrier-function guidance prevents collisions without retraining

CoDiG modifies the reverse SDE of diffusion by adding a barrier-function gradient at each denoising step: `score_modified = score_learned - γ_t · ∇_x V(x_t; C)` where V is a barrier function that becomes infinite at constraint boundaries. The time-varying weight γ_t increases during denoising, so constraints are enforced most strongly in the final (cleanest) steps. For the user's problem, V encodes obstacle proximity computed directly from point cloud data as a signed distance field. (arXiv) (themoonlight)

The critical advantage is that **barrier guidance is injected at inference only** — the score model trains on unconstrained demonstrations, and safety is overlaid post-hoc. This means the user's existing iDP3 training pipeline remains unchanged. The barrier function V(x; C) for UAV obstacle avoidance takes the form `V = -Σ log(||p_i - p_obs||² - r_safe²)` where p_obs are nearest obstacle points from the point cloud and r_safe is the safety radius. Computing ∇_x V requires one backward pass through the trajectory-to-distance mapping — adding **~2–5ms per denoising step**, acceptable for 10–20 Hz UAV control. (arXiv) (openreview)

CoDiG also introduces a **warm-start mechanism**: instead of initializing each planning cycle from noise, it perturbs the previous plan with small noise and re-denoises. This provides temporal consistency (preventing the jittery re-planning that causes the oscillation-around-goal behavior) and cuts denoising steps further. Integration effort is **medium** — the barrier function is a pure Python/PyTorch addition to the denoising loop, implementable in ~200 lines. The main

engineering challenge is efficient SDF computation from streaming point clouds. themoonlight arXiv

5c. Paper #20's point-cloud-conditioned energy diffusion is the closest architectural match

The IEEE RA-L 2025 paper "Real-Time Adaptive Motion Planning via Point Cloud-Guided, Energy-Based Diffusion and Potential Fields" (2507.09383) is the single most architecturally relevant work. It uses a **PointNet + Set Transformer** encoder to process obstacle point clouds into a 320-dimensional latent vector, which conditions a temporal U-Net diffusion model via cross-attention. During DDIM sampling, **Artificial Potential Field gradients** are injected alongside the learned score, combining data-driven trajectory generation with analytical obstacle avoidance. arXiv + 2

This architecture maps almost directly onto the user's setup. The key differences: Paper #20 uses a temporal U-Net (iDP3 uses a 1D temporal U-Net with a different point cloud encoder), and it was **trained on an RTX 2080 Ti** (11GB), proving feasibility on hardware far weaker than the user's 24GB GPU. The APF component computes repulsive forces from point cloud obstacle positions and attractive forces toward the goal — exactly the physics needed for UAV navigation. Training used **2,000 obstacle configurations × 20 start-goal pairs × 25 trajectories per pair** (~1M samples), suggesting the user's 200 episodes are an order of magnitude too few for this approach and confirming the need for Phase 0's procedural world generation. arXiv arXiv

Integration effort is **high** because it requires replacing iDP3's point cloud encoder with a PointNet + Set Transformer and restructuring the conditioning mechanism. However, the paper provides a complete architectural template. A pragmatic alternative: keep iDP3's encoder but add Paper #20's APF gradient injection during denoising. This hybrid would be **medium** effort and capture most of the safety benefit.

5d. DPPO enables structured RL fine-tuning that stays on-manifold

DPPO (ICLR 2025, Princeton) reformulates diffusion policy fine-tuning as a **two-layer MDP** where the inner MDP is the denoising chain itself. PPO is applied to both layers, but critically, **only the last K' denoising steps are fine-tuned** (e.g., last 5 of 100), keeping early steps frozen. This provides "structured, on-manifold exploration" — the policy explores variations of reasonable trajectories rather than random actions, preventing catastrophic forgetting of the base imitation-learned behavior. arXiv arXiv

For the user's Phase 2, DPPO is the natural choice over vanilla PPO because the iDP3 policy already shows partial avoidance behavior. A reward function targeting the specific failure modes — `R = w1·goal_progress + w2·min_obstacle_clearance - w3·collision - w4·|yaw_rate| - w5·|z - z_target|` — would directly penalize the 336° yaw and altitude crash. DPPO runs **40–50 parallel environments on CPU** with a single GPU for policy updates; the user would run 4–8 parallel Gazebo instances (constrained by CPU/RAM rather than GPU). The 24GB GPU is sufficient — DP3 training uses ~10GB, and DPPO's overhead is primarily in rollouts. arXiv GitHub

The DPPO codebase (github.com/irom-princeton/dppo) provides a clean PyTorch implementation. Adapting it requires: writing a Gazebo Gym wrapper (already planned as Phase 1b), defining the reward function, and connecting the iDP3 policy. Engineering effort for the wrapper is **medium**; DPPO integration itself is **low-medium** given the clean codebase.

5e. Classifier-free guidance is the simplest fix for goal-proximity instability

While not on the original verification list, classifier-free guidance (CFG) from BESO (RSS 2023, arXiv 2304.02532) is the **lowest-effort, highest-impact** modification for the 2.24× goal-proximity variance. During training, the goal condition is randomly dropped with probability p=0.1–0.2. At inference, the guided score becomes `ε_guided = ε_uncond + λ·(ε_cond - ε_uncond)` where $\lambda > 1$ amplifies goal-seeking behavior. Near the goal, increasing λ to 2.0–3.0 forces the model to commit to a single goal-approaching mode rather than averaging across multiple approach trajectories. (arXiv) (arXiv)

Implementation requires ~**50 lines of code**: a random dropout mask on goal conditioning during training, and a second forward pass (unconditional) during inference. The computational cost doubles inference time per step but with truncated diffusion (2 steps), this means 4 forward passes total — still well under real-time budget. CFG directly converts the multi-modal distribution near the goal into a unimodal one, eliminating the oscillation behavior. Combined with DiffusionDrive's anchors, CFG provides **belt-and-suspenders** mode control: anchors handle obstacle-avoidance mode selection, while CFG handles goal-approach mode locking.

---

6. v8.1 upgrade proposal

The v8.1 design retains the three-phase structure but makes specific changes informed by the verified works. The most significant architectural decision is adding **three inference-time mechanisms** to the denoising loop — anchored initialization, barrier gradients, and classifier-free guidance — rather than relying solely on learned behavior from demonstrations.

Phase 0 (unchanged scope, refined targets)

Environment diversification: Generate **50+ procedurally randomized Gazebo worlds** using Python scripts that vary obstacle count (3–30), obstacle geometry (cylinders, boxes, walls, L-shapes), corridor widths, and goal positions. Paper #20's training used 2,000 configurations; targeting 100–200 configurations with 10–20 EGO planner episodes each yields **1,000–4,000 episodes** — a 5–20× improvement over the current 200 that is the minimum viable dataset for navigation generalization.

Point cloud augmentation (informed by PointFlowMatch and PCWM literature): Add random SE(3) rotation around gravity axis (±30°), Gaussian jitter (σ=0.02m), random 20% point dropout, and FPS seed variation per epoch. Apply the same SE(3) transform to action labels. The 95.69% action change on point cloud zeroing indicates dangerous overfitting to specific point patterns — dropout and jitter will regularize this.

NEW — Anchor extraction: Cluster all training trajectories (after data expansion) into **K=8 prototypes** via K-means in (Δx, Δy, Δz, Δyaw) space. Store these as the anchor bank for Phase 1a.

NEW — Normalizer scope expansion: Update the normalizer to compute running statistics per-anchor rather than globally, preventing cross-mode averaging in action normalization.

Phase 1a (major revision: anchored truncated diffusion + CFG + barrier guidance)

Replace the current full-chain denoising with a **three-mechanism denoising loop**:

1. **Anchor selection**: A lightweight MLP head scores each of K=8 anchors based on the point cloud encoding. Select the top-scoring anchor as initialization: `x_init = anchor_best + σ·ε` with σ corresponding to ~5% of the full noise schedule.
2. **Truncated denoising with CFG**: Run **3–5 DDIM steps** (not 100 DDPM steps) from x_init. At each step, compute guided score: `ε_guided = ε(x_t, ∅) + λ·(ε(x_t, goal) - ε(x_t, ∅))`. Set λ=1.5 globally, increasing to λ=2.5 when goal distance < 2m. Train with 15% goal-condition dropout.
3. **CoDiG-style barrier gradient**: At each denoising step, compute signed distance to nearest obstacle points from the point cloud. Add barrier gradient: `x_t ← x_t - γ_t · ∇_x V(x_t)` where V is the log-barrier on obstacle clearance. Apply altitude floor (z ⩾ 1.0m) and yaw-rate ceiling (|Δyaw| ⩽ 30°/step) as hard barrier constraints.

Nominal trajectory: Instead of DiffusionDrive's data-driven anchors only, add a **"straight-to-goal" anchor** computed analytically at each timestep. This anchor always exists as option 0, ensuring the model has a direct goal-approach trajectory to refine. Near the goal in obstacle-free space, this anchor dominates, naturally reducing variance — directly targeting the 2.24× goal-proximity prediction std.

Training recipe change: Train the anchor selection head end-to-end with the diffusion model. Use the EGO planner expert's trajectory as ground truth; the training loss adds a cross-entropy term for anchor classification alongside the standard diffusion L2 loss.

Phase 1b (unchanged: Gazebo Gym wrapper)

Build the wrapper with DPPO-compatible interface: `step()` returns (obs, reward, done, info) where obs includes point cloud + agent_pos + goal_pos. Reward function: `R = 2.0·Δgoal_dist + 5.0·min(obstacle_clearance, 1.0)/1.0 - 50.0·collision - 10.0·(|yaw_rate| > 45°/s) - 20.0·(z < 1.0m) + 100.0·goal_reached`. Support 4–8 parallel Gazebo instances.

Phase 2 (conditional branching, unchanged logic but refined criteria)

If Phase 1a achieves ⩾70% success rate in unseen single-obstacle worlds: Apply CoDiG barrier guidance more aggressively (increase γ_t) and add NavDP-style critic scoring. Train a small critic network using Gazebo's octomap as privileged ESDF information to score candidate trajectories, selecting the safest among 4–8 diffusion samples per timestep. This is the "geometric safety filter" path. (arXiv)

If Phase 1a achieves <70% success rate: Deploy DPPO fine-tuning with the Phase 1b Gym wrapper. Fine-tune only the last 3 denoising steps (of 5 total), keeping the anchor selection and point cloud encoder frozen. Run for 500–1000 PPO iterations across 4 parallel Gazebo environments. The structured exploration from DPPO's two-layer MDP formulation ensures the policy refines within the manifold of reasonable UAV trajectories. (arXiv + 2)

NEW decision criterion — elevating goal-proximity stability: Add a specific gate metric: **goal-proximity prediction std must be ≤1.5× GT std** before declaring Phase 1a/2 success. This makes the previously tertiary concern a primary pass/fail criterion.

Phase 3 (NEW — deployment optimization)

After the policy works reliably: distill to a **ManiCM consistency model** for single-step inference (~1–2ms). ManiCM is already built on the DP3 codebase with point cloud input, requiring minimal adaptation. Training takes ~4 hours on 24GB GPU. This eliminates residual multi-step mode switching and achieves the inference speed needed for real drone deployment. Apply the CBF safety layer (altitude, yaw-rate, obstacle clearance) as a hard post-processing filter on the consistency model output. (alphaXiv)

---

7. What will NOT help despite looking promising

DroneDiffusion learns quadrotor *dynamics* (predicting next state from current state + action), not trajectory *planning*. It's a world model, not a policy. Integrating it would require building an MPC loop on top — a fundamentally different architecture from the user's imitation-learning approach. The engineering effort is high and the benefit is orthogonal to the core failures. (arXiv +2)

SwarmDiffusion uses RGB images with FiLM conditioning, not point clouds. Adapting its embodiment-agnostic framework to point cloud input would require re-engineering the entire perception pipeline. The cross-embodiment transfer capability is impressive but irrelevant — the user has one embodiment (a quadrotor). (arXiv + 2)

Leapfrog Diffusion Model predicts where pedestrians will walk by accelerating the diffusion process through a "leapfrog" initializer. While the speed-up concept is related to truncated diffusion, the model operates on 2D bird's-eye-view coordinates of pedestrians — it cannot generate robot control actions and has no mechanism for obstacle avoidance.

DEAL (NeurIPS 2025) addresses sim-to-real via domain randomization using diffusion-based adversarial perturbations. While sim-to-real transfer matters for the user's final goal, DEAL's approach requires training a separate adversarial generator — high complexity for uncertain benefit when the core policy doesn't yet work in simulation. (NeurIPS)

Constrained Bridged Diffusion (AAAI 2025) and **Aligning Diffusion with Constraints** (2504.00342) both offer elegant mathematical frameworks for constrained generation, but their

evaluations are limited to 2D toy problems and tabletop manipulation. The gap to 3D UAV navigation is too wide for practical transplantation.

DiffuserLite's hierarchical diffusion is designed for long-horizon planning in D4RL mazes — useful conceptually but operates on state sequences, not action sequences with point cloud conditioning. The DiT backbone idea is worth noting but not worth the architectural overhaul.
NeurIPS

---

8. Key verified papers with reliable citation information

Directly actionable for v8.1:

- **DiffusionDrive**: Liao et al., "Truncated Diffusion Model for End-to-End Autonomous Driving," CVPR 2025 Highlight. arXiv:2411.15139. Code: github.com/hustvl/DiffusionDrive
- **CoDiG**: Ma et al., "Constraint-Aware Diffusion Guidance for Robotics: Real-Time Obstacle Avoidance for Autonomous Racing," CoRL 2025. arXiv:2505.13131
- **DPPO**: Ren et al., "Diffusion Policy Policy Optimization," ICLR 2025. arXiv:2409.00588. Code: github.com/irom-princeton/dppo
- **SafeDiffuser**: Xiao et al., "Safe Planning with Diffusion Probabilistic Models," ICLR 2025. arXiv:2306.00148. Code: github.com/Weixy21/SafeDiffuser
- **NavDP**: Cai et al., "Learning Sim-to-Real Navigation Diffusion Policy with Privileged Information Guidance," 2025. arXiv:2505.08712. Code: github.com/wzcai99/NavDP
- **PC-Guided Energy Diffusion**: Teshome et al., "Real-Time Adaptive Motion Planning via Point Cloud-Guided, Energy-Based Diffusion and Potential Fields," IEEE RA-L 2025. arXiv:2507.09383
- **DPCC**: Römer et al., "Diffusion Predictive Control with Constraints," L4DC 2025. arXiv:2412.09342. Code: github.com/ralfroemer99/dpcc

UAV-specific verified works:

- **DroneDiffusion**: Das et al., "Robust Quadrotor Dynamics Learning with Diffusion Models," ICRA 2025. arXiv:2409.11292. Code: github.com/avirupdas55/DroneDiffusion
- **SketchPlan**: Norelius et al., "Diffusion Based Drone Planning From Human Sketches," IEEE RA-L 2025. arXiv:2510.03545. Code: github.com/sixnor/SketchPlan
- **ImpedanceDiffusion**: Batool et al., "Diffusion-Based Global Path Planning for UAV Swarm Navigation with Generative Impedance Control," 2026. arXiv:2603.09031
- **SwarmDiffusion**: Zhura et al., "End-To-End Traversability-Guided Diffusion for Embodiment-Agnostic Navigation," 2025. arXiv:2512.02851

Background/foundational (not on original list but essential for v8.1):

- **iDP3**: Ze et al., "Generalizable Humanoid Manipulation with Improved 3D Diffusion Policies," IROS 2025. arXiv:2410.10803. Code: github.com/YanjieZe/Improved-3D-Diffusion-Policy
- **ManiCM**: Lu et al., "ManiCM: Real-time 3D Diffusion Policy via Consistency Model for Robotic Manipulation," 2024. arXiv:2406.01586. Code: github.com/ManiCM-fast/ManiCM
- **BESO** (CFG for robot policies): Reuss et al., RSS 2023. arXiv:2304.02532. Code: github.com/intuitive-robots/beso

---

Conclusion

The verification exercise exposes a counterintuitive finding: the surveys are mostly accurate on paper existence but deeply misleading on applicability. Only 3 of 25 real papers operate in a domain close enough to borrow from directly (DiffusionDrive for truncated diffusion, CoDiG for barrier guidance, Paper #20 for point-cloud-conditioned planning). The remaining 22 require non-trivial domain adaptation.

The user's two failure modes — obstacle ignorance in unseen worlds and multi-modal collapse near goals — have distinct solutions that can be layered. **Anchor-based truncated diffusion** constrains the output space to reasonable trajectory neighborhoods, preventing the 336° yaw excursion. **Classifier-free guidance** with goal conditioning locks the model to a single approach mode near the goal, directly targeting the 2.24× variance. **CoDiG barrier gradients** enforce obstacle clearance analytically during denoising, providing a safety floor independent of learned behavior. These three mechanisms are additive, not competitive — implementing all three in the denoising loop is the recommended v8.1 architecture.

The most important insight from this analysis is that **the user's problem is not primarily architectural** — iDP3 is adequate as a backbone. The failures stem from insufficient data diversity (200 episodes from one world), missing inference-time safety mechanisms, and no explicit mode-selection strategy. The v8.1 upgrade addresses all three without requiring a full architecture replacement, keeping the engineering scope realistic for a single-GPU, ROS1/PX4/Gazebo stack.